Universidade Federal de Juiz de Fora
DCC059 - Teoria dos Grafos
Especificação do Trabalho 2

Motivação

Durante a disciplina de grafos, foram apresentados os principais conceitos e algoritmos de Teoria dos Grafos. Os algoritmos estudados em sala (Prim, Kruskal, Dijkstra e Floyd-Warshall, por exemplo) são amplamente conhecidos por serem capazes de fornecer, de forma eficiente, uma solução exata para os respectivos problemas.

Entretanto, muitos problemas envolvendo grafos são problemas de Otimização Combinatória considerados intratáveis, isto é, problemas para os quais não são conhecidos algoritmos capazes de encontrar a solução ótima em tempo polinomial. Estes problemas são frequentemente resolvidos por algoritmos heurísticos, que têm como objetivo encontrar boas soluções de forma rápida.

Neste trabalho, os alunos deverão desenvolver um projeto de programação relacionado a um problema intratável de Otimização Combinatória. Além de colocar em prática os conhecimentos adquiridos na disciplina, os alunos precisarão organizar, comparar e analisar os resultados obtidos para diferentes instâncias do problema, fornecendo um relatório detalhado do projeto.

Objetivos

O objetivo geral deste trabalho é que cada grupo de alunos desenvolva um projeto de implementação para resolver heuristicamente um problema clássico de grafos. Espera-se que os alunos, além da implementação, trabalhem na execução e comparação de experimentos, na análise de resultados de algoritmos e na confecção de um relatório sobre o projeto.

Desenvolvimento do trabalho

O trabalho requer uma implementação e um relatório de desenvolvimento. Além disso, parte das questões da 3ª avaliação (prova) será sobre o trabalho de cada grupo. Cada um destes itens é detalhado a seguir.

Projeto de implementação

O projeto de implementação deverá ser desenvolvido **na linguagem C++** e deverá incluir:

- um tipo abstrato de dados Grafo, armazenando nós e arestas/arcos em uma estrutura de dados de sua escolha, de forma a assegurar que represente grafos conforme sejam definidas as instâncias do problema (direcionados/não direcionados, ponderados nas arestas, ponderados nos vértices etc.);
- geração de semente de randomização baseada em data/hora (chamada uma única vez em todo o código) e impressão desta para que seja possível repetir um teste

específico, caso necessário (também é interessante permitir que a semente seja informada como um parâmetro de execução);

- leitura dos dados de arquivo, no formato padrão das instâncias do problema na literatura;
- um algoritmo guloso para o problema;
- um algoritmo guloso randomizado, em que o valor de α e o número de iterações sejam parâmetros;
- um algoritmo guloso randomizado reativo, em que os valores de α, o número de iterações e o tamanho dos blocos sejam parâmetros;
- um **verificador** de soluções, independente dos algoritmos, que receba o grafo e uma solução e confira se a solução é viável (por exemplo: se é uma árvore geradora, se usa apenas arestas do grafo, se tem $|V| - 1$ arestas, se é conexa) e se o valor reportado da função objetivo está correto, recalculando-o;
- impressão da solução (em arquivo ou na tela), preferencialmente em formato que facilite sua visualização (permitindo copiar e colar em http://csacademy.com/app/graph_editor/, por exemplo).

Para facilitar a organização dos resultados obtidos, sugere-se que, no final da execução de cada algoritmo, o código acrescente uma linha em um arquivo `.csv` contendo os dados da execução. Neste caso, o ideal é que os dados incluam:

- data/hora do teste;
- instância;
- eventuais parâmetros do teste, dependendo do problema (capacidades, limites máximos, cardinalidade máxima etc.);
- algoritmo executado no teste;
- parâmetros do algoritmo (α, número de iterações, tamanho do bloco);
- semente de randomização;
- tempo gasto na execução;
- valor da melhor solução alcançada.

Embora nem todos os algoritmos apresentem todos esses dados, construir uma tabela única (em que algumas colunas podem estar vazias nas linhas correspondentes ao guloso, por exemplo) ajuda bastante na hora de consolidar os resultados. Além dos dados listados, o α que forneceu a melhor solução do algoritmo reativo e a média dos valores das soluções obtidas em todas as iterações dos algoritmos randomizados, por exemplo, também podem gerar resultados interessantes no final.

Tanto a corretude da implementação quanto a organização e a clareza do código serão avaliadas. Fiquem à vontade para criar classes auxiliares e funções adicionais que contribuam para um código bem estruturado, organizado e de fácil compreensão. O código deve estar compilando e executando (**se não compilar, a nota é zero**).

Será incluída uma atividade no Classroom para que os alunos informem o link do repositório git em que o código estará disponível. No diretório raiz do repositório, insira um arquivo `.txt` com uma descrição mínima de como executar o programa via linha de comando (em ambiente Linux), informando como executar cada algoritmo (linha de comando e parâmetros).

Experimentos

Para os experimentos, cada grupo deve escolher três valores de α, de acordo com o que for mais adequado para o seu algoritmo. Os valores apresentados no modelo de relatório são apenas exemplos ilustrativos.

Cada algoritmo deve ser executado **10 vezes para cada instância e para cada valor de** α, cada vez com uma semente diferente, registrando ao menos a melhor solução e o tempo médio de processamento (em segundos). No algoritmo guloso randomizado, o algoritmo construtivo deve ser chamado pelo menos 30 vezes. No algoritmo guloso randomizado reativo, o construtivo deve ser chamado pelo menos 300 vezes, com blocos de 30 a 50 iterações para possibilitar a atualização das probabilidades.

Todas as soluções utilizadas nas tabelas de resultados devem ter sido aprovadas pelo verificador. Cabe a cada grupo avaliar o tempo que os experimentos demandam, de forma a assegurar que as entregas sejam feitas até a data estabelecida.

Relatório

O relatório do trabalho deverá seguir o modelo disponibilizado no Classroom. O referido modelo especifica o que deve fazer parte do relatório, incluindo exemplos de como algoritmos e tabelas devem ser inseridos. O relatório deve conter até 10 páginas (sem contabilizar capa, sumário e bibliografia) e **não** deve conter código-fonte.

O relatório deve apresentar os testes de verificação da implementação:

- a descrição do verificador implementado;
- testes em instâncias pequenas construídas pelo grupo, cuja solução ótima tenha sido obtida à mão;
- a execução do algoritmo guloso, passo a passo, em uma dessas instâncias.

No relatório, devem ser inseridas, no mínimo, as seguintes tabelas de resultados:

- desvios percentuais relativos às melhores soluções obtidas por cada algoritmo para cada instância, em relação à melhor solução conhecida da respectiva instância (este resultado corresponde ao melhor resultado alcançado em uma das 10 execuções);
- desvios percentuais relativos à média das melhores soluções obtidas em cada execução do algoritmo para cada instância, em relação à melhor solução conhecida da respectiva instância (aqui, é utilizada a média dos resultados das 10 execuções);
- o tempo médio das 10 execuções de cada algoritmo.

No Classroom, foi disponibilizada uma planilha para ajudar no cálculo das tabelas de desvio percentual. Preenchendo os dados obtidos, a planilha calcula os desvios percentuais e formata as linhas para colar no LaTeX.

Uso de ferramentas de inteligência artificial

O objetivo do trabalho é que os alunos aprendam a projetar e a implementar uma heurística. Código gerado por ferramentas de inteligência artificial (IA) pode compilar e, ainda assim, estar errado de formas sutis: gerar soluções inviáveis, calcular a função objetivo incorretamente ou implementar a randomização de forma inadequada. Por isso, caso o grupo utilize ferramentas de IA:

- o grupo é responsável por **todo** o conteúdo entregue: código, resultados e texto;
- o código deve ser testado, conforme descrito nas seções anteriores (verificador e instâncias pequenas com ótimo conhecido);
- o relatório deve conter uma seção curta descrevendo como a IA foi utilizada (ou declarando que não foi utilizada).

Cada aluno deve ser capaz de explicar, modificar e executar à mão qualquer parte do trabalho do seu grupo.

Avaliação

A entrega do trabalho (código e relatório) vale **5 pontos**. A 3ª avaliação (prova) vale **30 pontos**, dos quais **15 a 20 pontos** serão referentes a questões sobre o trabalho prático; as demais questões podem incluir outros conteúdos da disciplina.

O trabalho é feito em grupo, mas todo aluno deve conhecer, entender e dominar todo o conteúdo e o desenvolvimento do trabalho. Por este motivo, recomenda-se que cada aluno participe de todas as etapas: implementação, testes, análise de resultados e escrita do relatório.

As questões da prova relacionadas ao trabalho prático serão automaticamente anuladas para os alunos que não participarem do trabalho. Da mesma forma, caso o aluno não consiga responder às questões da prova relacionadas ao trabalho, a nota de entrega do trabalho será zerada. Isto é, as notas relativas ao trabalho são dependentes uma da outra.

Datas

A entrega do trabalho (código e relatório) e a 3ª avaliação (prova) ocorrerão em **8 de dezembro de 2026**.

Temas

Cada trabalho deve se concentrar em um dos temas especificados abaixo. Os grupos deverão indicar em formulário sua preferência, mas a atribuição final será realizada pela professora e disponibilizada no Classroom. A especificação completa de cada tema será enviada a cada grupo depois da atribuição.

Os dois temas são variações do problema da árvore geradora e são problemas NP-difíceis, isto é, não se conhecem algoritmos exatos eficientes para resolvê-los.

A. Árvore geradora com máximo de folhas

Definição. Dado um grafo $G = (V, E)$ conexo e não ponderado, o problema consiste em encontrar uma árvore geradora T de G com o maior número possível de folhas, isto é, de vértices de grau 1 em T:

$$\max \ |\{\, v \in V : \text{grau}_T(v) = 1 \,\}|.$$

Aplicação. Redes de sensores, por exemplo para a detecção de incêndios florestais: os vértices internos da árvore formam a espinha dorsal da rede.

B. Árvore geradora com mínimo de vértices de ramificação

Definição. Um vértice de uma árvore T é de *ramificação* se tem grau maior que 2 em T. Dado um grafo $G = (V, E)$ conexo, o problema consiste em encontrar uma árvore geradora T de G com o menor número possível de vértices de ramificação:

$$\min \ |\{\, v \in V : \text{grau}_T(v) > 2 \,\}|.$$

Aplicação. Redes ópticas, em que os vértices de ramificação exigem equipamentos (divisores de sinal) de custo elevado.

Dúvidas

O trabalho é em grupo e deve ser feito sempre segundo as orientações. Leve à professora as decisões de projeto para serem discutidas, sempre que necessário. Em caso de dúvidas, envie mensagem via Classroom.